CHAPTER 1

Introduction to Discrete Mathematics

Notes By Pr. El Hadiq Zouhair

1.1 What is discrete mathematics?

Discrete mathematics is the study of mathematical structures that are fundamentally *discrete* rather than continuous: integers, finite sets, graphs, logical statements, sequences and algorithms. Where calculus deals with quantities that vary smoothly, discrete mathematics deals with objects that can be counted, listed, and manipulated one at a time.

Definition 1.1. *Discrete mathematics* is the branch of mathematics dealing with countable structures — objects that can assume only distinct, separated values — as opposed to continuous structures such as the real line.

The natural habitat of discrete mathematics is the world of integers $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ and of finite collections. A digital computer is itself a discrete object: its memory holds finitely many bits, each either 0 or 1, so every question about computation is ultimately a question of discrete mathematics.

1.2 The main questions

Across all its subfields, discrete mathematics keeps returning to a small number of fundamental questions.

- **Existence.** Is there an object with a given property? (Is there a path visiting every vertex of a graph exactly once?)
- **Counting.** How many such objects are there? (How many ways can a committee of 5 be chosen from 30 students?)
- **Optimization.** Which object is best? (Which spanning tree of a network has minimal total cost?)
- **Structure.** How are the objects organised? (How does a partial order arrange the subsets of a set?)

- **Algorithms.** How can the answer be computed, and how fast? (Can satisfiability of a logical formula be decided efficiently?)

1.3 Information, summarization and computation

Three recurring activities motivate the tools we will build.

1.3.1 Information

Discrete structures *encode information*. A truth table encodes the behaviour of a logical circuit; a set encodes membership; a relation encodes links between objects; a binary string encodes a file. Understanding which structures can encode which information — and how compactly — is a central concern.

1.3.2 Summarization

Mathematics compresses infinitely many facts into a single statement. The formula

$$\sum_{i=1}^{n} i \;=\; \frac{n(n+1)}{2}$$

summarises infinitely many arithmetic identities at once. Quantifiers ($\forall$, $\exists$), closed forms and recurrences are all instruments of summarization, and proving such summaries correct is what Part IV of this book is about.

1.3.3 Computation

Every algorithm is a discrete process: a sequence of elementary steps acting on discrete data. Counting the steps (complexity), proving the result correct (invariants, induction) and even defining what "computable" means are tasks for discrete mathematics.

1.4 A brief history

Discrete themes are as old as mathematics itself. Euclid (c. 300 BCE) proved that there are infinitely many primes — a discrete existence theorem we shall reprove in Chapter 12. Combinatorial counting flourished with Pascal and Fermat in the 17th century around games of chance, producing Pascal's triangle (Chapters 10–11). Euler's 1736 solution of the Königsberg bridge problem founded graph theory. In the 19th century, Boole turned logic into algebra and Cantor created set theory, giving the subject its modern foundations (Parts I–II). The 20th century brought the decisive turn: Gödel, Turing and Church connected logic to computation, and the arrival of digital computers transformed discrete mathematics from a collection of curiosities into the mathematical backbone of computer science.

1.5 Discrete mathematics today

Modern applications are everywhere:

- **Computer science** — data structures, databases, programming language semantics, verification of software and hardware.
- **Cryptography** — number theory and finite algebra protect every online transaction.

- **Networks** — graph theory models the internet, social networks and transportation systems.
- **Data science** — counting, probability on finite spaces, and combinatorial optimisation underlie machine learning pipelines.
- **Operations research** — scheduling, allocation and routing are discrete optimisation problems.

> *Continuous mathematics asks "how much?"; discrete mathematics asks "how many?", "does one exist?", and "how can we build it?".*

1.6 Why study discrete mathematics?

Beyond its applications, discrete mathematics teaches a craft: the craft of *precise reasoning*. In this course you will learn to

- read and write statements whose meaning is completely unambiguous (Chapters 2–5);
- manipulate the basic structures of all mathematics — sets, relations, functions, sequences (Chapters 6–9);
- count complicated configurations exactly (Chapters 10–11);
- construct proofs that withstand any scrutiny, including proofs by induction and recursive constructions (Chapters 12–14).

These skills transfer directly to programming, to algorithm design and to every mathematical subject you will meet later.

1.7 About this course

The book is organised in four parts.

- **Part I — Foundations of Logic.** Propositions, connectives, truth tables, logical equivalence, predicates, quantifiers and rules of inference: the language in which all later chapters are written.
- **Part II — Discrete Structures.** Sets and their operations, relations, functions, sequences and series: the objects that the language talks about.
- **Part III — Counting.** Permutations, combinations, the binomial theorem and a toolbox of binomial identities.
- **Part IV — Proofs and Recursion.** Proof techniques, mathematical induction in its weak, strong and structural forms, and recursive definitions of functions, sets and structures.

Each chapter ends with a graded set of exercises — tagged EASY, MEDIUM or HARD — followed by complete solutions.

1.8 Who is this course for?

The course assumes no prerequisites beyond secondary-school algebra. It is designed for first-year students of computer science, mathematics, engineering and data science, and for anyone who wants to reason precisely about discrete problems. Mathematical maturity is not required at the start — building it is precisely the point.

Exercises

Exercise 1.1 EASY

Classify each of the following as a *discrete* or a *continuous* question: (a) How many 5-card poker hands are possible? (b) What is the area under the curve $y = x^2$ between 0 and 1? (c) Is there a way to schedule 10 exams in 4 slots with no student conflict? (d) At what speed does a falling object hit the ground?

Exercise 1.2 EASY

Give one example from everyday life of each of the five main questions of discrete mathematics (existence, counting, optimization, structure, algorithms).

Exercise 1.3 MEDIUM

A byte consists of 8 bits, each 0 or 1. How many distinct bytes are there? More generally, how many distinct bit strings of length n are there? Justify your count.

Exercise 1.4 MEDIUM

The formula $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$ "summarises infinitely many facts". Verify it directly for $n = 1, 2, 3, 4$, and explain why no amount of direct verification constitutes a proof for all n.

Exercise 1.5 HARD

Euclid proved there are infinitely many primes. Suppose, for contradiction, that $p_1, p_2, \ldots, p_k$ were *all* the primes, and consider $N = p_1 p_2 \cdots p_k + 1$. Show that none of the p_i divides N, and deduce a contradiction. (You may use the fact that every integer greater than 1 has at least one prime divisor.)

Solutions to the Exercises

Solution 1.1. (a) Discrete — a finite count. (b) Continuous — an integral over a real interval. (c) Discrete — existence of a finite arrangement. (d) Continuous — a real-valued physical quantity varying smoothly with time. ■

Solution 1.2. For instance: *existence* — is there a seating plan where no two rivals sit together? *Counting* — how many phone numbers are possible with a given prefix? *Optimization* — which route to work is shortest? *Structure* — how is a family tree organised? *Algorithms* — what procedure sorts a list of names alphabetically? (Any sensible examples are acceptable.) ■

Solution 1.3. Each of the 8 bits can be chosen independently in 2 ways, so there are $2 \cdot 2 \cdots 2 = 2^8 = 256$ bytes. The same argument gives 2^n bit strings of length n: choosing a string means making n independent binary choices, and by the product rule (Chapter 10) the total is 2^n. ■

Solution 1.4. $n = 1$: $1 = \frac{1 \cdot 2}{2}$. $n = 2$: $1 + 2 = 3 = \frac{2 \cdot 3}{2}$. $n = 3$: $1 + 2 + 3 = 6 = \frac{3 \cdot 4}{2}$. $n = 4$: $1 + 2 + 3 + 4 = 10 = \frac{4 \cdot 5}{2}$. Direct verification only checks finitely many instances, while the claim is about *all* natural numbers n; no finite list of checks can rule out a failure at some larger n. A general argument — such as induction (Chapter 13) or Gauss's pairing trick — is required. ■

Solution 1.5. Suppose $p_1, \ldots, p_k$ are all the primes and let $N = p_1 p_2 \cdots p_k + 1$. For each i, dividing N by p_i leaves remainder 1, since $p_i \mid p_1 \cdots p_k$; hence $p_i \nmid N$. But $N > 1$, so N has some prime divisor q. Then q is a prime different from every p_i, contradicting the assumption that the list contained all primes. Therefore there are infinitely many primes. ■